COLLEGE OF COMPUTER AND INFORMATION SCIENCE

Academic Year 2025 – 2026

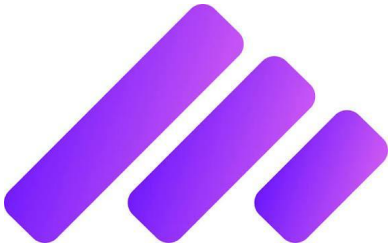
Practicum Engagement Narrative: Prospera

John Aldrine F. Lim

Adviser: Dennis A. Martillano

Submitted to the Faculty of Mapúa Malayan Colleges Laguna
In Partial Fulfillment of the Requirements for the degree of

Bachelor of Science in Computer Science



PRACTICUM NARRATIVE REPORT

On-the-Job Training Engagement
Prosperna Inc.

1. Overview of the Practicum Engagement

1.1 Company Background

Prosperna is a Philippine technology and software company focused on helping small and medium-sized businesses establish and grow their online presence. The company operates primarily as an eCommerce Software-as-a-Service (SaaS) platform, providing tools that allow businesses to create online stores, manage products and orders, accept payments, coordinate shipping, and engage with customers. Prosperna positions itself as a homegrown Philippine technology company built specifically for Philippine businesses. Its platform is designed around the particular needs of Filipino entrepreneurs and MSMEs, including local payment methods, cash-on-delivery options, local courier integrations, and tools for selling online without requiring extensive technical or coding knowledge.

1.2 Nature of Assignment and Tasks

During this 9-week practicum engagement, contributions were made across multiple divisions and repositories as a Full Stack Software Engineer. The primary focus areas included:

- Pippify Web Form System: Design and implementation of form builder UI, public form interface, and lead creation pipeline integration
- AINA MaxAI Platform: Development of conversation management features, domain chat improvements, blog agent implementation, and Max Marketing widget creation
- Business Profile Service: Conversation list pagination, caching mechanisms, and performance optimization
- Content Calendar System: Persistent calendar with full-page editor and market service integration

- Marketing Features: Multi-email carousel, captions generation for social media, widget refinement
- Orders System: Abandoned cart detection and reminder notification implementation
- Navigation & UX: Dashboard navigation improvements, per-agent hints, and overall user experience enhancements
- Code Quality: Bug fixes, merge conflict resolution, test improvements, and comprehensive documentation

Metric	**Value**
Training Period	May 18 - July 25, 2026
Total Weeks	10 weeks

2. Presentation of Output

2.1 Summary of Contributions

Over the 10-week engagement, Multiple commits were submitted across 8 repositories, implementing features, fixing bugs, and improving system performance. The work spanned from backend API development to React frontend implementation, database schema design, and system integration.

2.2 Repositories and Commit Distribution

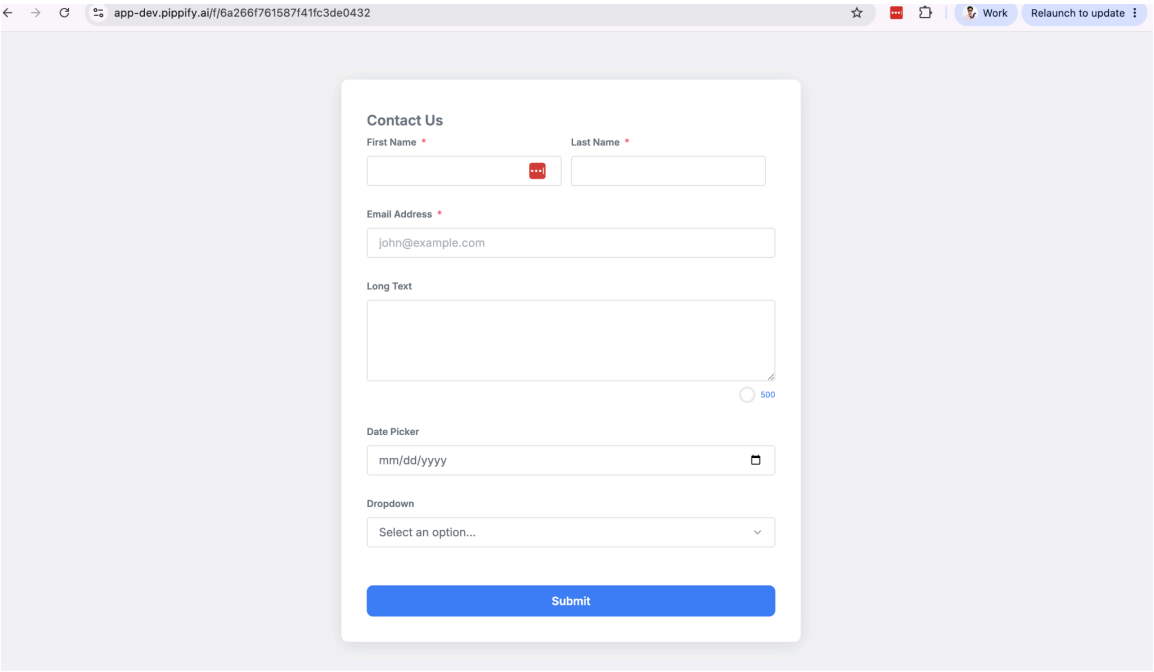
Repository	Commits	Primary Work
max-ai-inbound-frontend	43	Web form builder UI, public interface, field management
max-ai-inbound-service-api	44	Form submission backend, lead pipeline, AI integration
business-profile-service	8	Conversation pagination, caching, performance
aina	60	Conversation management, blog agent, MaxAI marketing widget
p1	22	Platform integration, blog keywords, navigation, Front end Fixes.
market	5	Content calendar, marketing service integration

orders	5	Abandoned cart detection, reminder system
email-service-api	2	Notification integration, email management

2.3 Key Features Implemented

Pippify Web Form System

Comprehensive form builder with field type support (text, long_text, first_name, last_name), public form interface with character limit visualization, form submission pipeline with AI lead scoring integration. Fixed critical bugs preventing webform submissions from appearing in inbox. Implemented proper lead creation flow with natural language processing for AI scoring.



This is the Client Side forms in which can be customizable by the Merchant side.

AINA MaxAI Platform Features

Implemented conversation list pagination and caching for improved performance. Fixed domain chat typing indicators. Developed blog agent features with keyword display and SEO field persistence. Created Max Marketing widget with email drafting, personalization options, and product promotion capabilities. Implemented multi-email carousel for browsing and saving email variations.

Max Marketing (New Sub Agent Implementation)

Max Marketing is a dedicated AI-driven marketing workspace inside Max AI, purpose-built for merchants who need to plan, write, and manage promotional content without hiring a marketer or designer. It lives inside a single side panel - the **Marketing panel** - with four tabs: **Brand Voice**, **Content Calendar**, **Email**, and **Captions**. A merchant opens it by simply asking Max in chat (e.g. "help me plan my content" or "write me a promo email"), and Max deep-links straight to the right tab, pre-fills it with on-brand content, and lets the merchant review, tweak, and save.

The system is built around one core principle: **the merchant never starts from a blank page.** Max always drafts first - an on-brand email, a month of content ideas, a set of captions - and the merchant edits, approves, and saves. Nothing is ever silently overwritten; multiple drafts can coexist, conflicts are flagged and confirmed before anything is replaced, and saved work is treated as durable rather than something that vanishes when the chat session ends.

1. Brand Voice - Teaching Max How Your Brand Talks

Before Max writes anything else, it needs to know how the merchant's brand sounds. This is the foundation every other feature builds on.

How a merchant triggers it: They simply ask, e.g. *"Set up my brand voice."* Max recognizes this and opens the Marketing panel directly on the Brand Voice tab.

What the merchant sees: A short form with a handful of fields - **Tone** (e.g. "Fresh, trustworthy, practical, and quality-first"), **Target audience** (e.g. "Small to mid-scale farmers"), **Words to use** (vocabulary do's), and **Words to avoid** (vocabulary don'ts, e.g. "cheap"). There's no jargon, no lengthy questionnaire - just plain-language fields a non-marketer can fill in confidently.

What happens on Save: Unlike almost every other tab in this system, Brand Voice is **not** session-only. When the merchant clicks Save, the values are persisted permanently to the merchant's business profile. This means brand voice survives forever - across sessions, across days, across completely new conversations with Max. Open a brand-new chat six months later, and Max still knows the tone.

How it flows into everything else: Every content-generating feature in the system - email drafting, caption writing, calendar planning - reads the saved brand voice and folds it into what it writes. The merchant sets it once; every downstream piece of content inherits it silently.

An important behavioral decision: Brand voice deliberately does **not** count as "unsaved work" for the purposes of locking the merchant into the Marketing agent. Early in development, brand voice was included in the same "is there unsaved content" check used to block switching to a different Max agent - which created a real bug: a merchant

who had properly saved their brand voice could *never* switch to another agent, because the check kept seeing brand voice fields as "content present" even though it was already safely persisted. This was fixed by excluding brand voice from the switch-lock entirely, since persisted preferences are fundamentally different from a live, unsaved draft. The lock now only cares about genuinely unsaved work: an email draft not yet cleared, or a calendar not yet saved.

2. Content Calendar - Planning a Full Month in Seconds

This is the most visually complex feature: a full month-view calendar grid that Max populates with content ideas across every channel the merchant uses.

How a merchant triggers it: *"Plan my content calendar for this month."* Max opens the panel on the calendar tab, optionally for a specific month (defaults to the current one).

Pre-loading existing plans: Before Max writes anything new, it checks whether the merchant already has a saved calendar for that month, pulling from the permanent store first and falling back to a temporary cache only if that's unreachable. If there are existing entries, Max tells the merchant what's already planned and which dates are already spoken for - a critical detail, because it sets up the conflict-avoidance behavior described next.

Generating entries: Max proposes a list of entries - each with a date, topic, channel (Instagram, Facebook, Email, TikTok, Blog, etc.), format, target audience, and a short content brief in the brand's voice. These render on a visual month grid, each day showing a compact chip with the channel's icon and the topic; hovering reveals the full brief.

Conflict detection - the safety net: If any of the newly-generated entries land on a date that already has saved content, the system does **not** silently overwrite it. Instead, it asks the merchant to choose: **"Replace those dates"** or **"Keep existing, skip conflicts."** Only after the merchant answers does Max proceed. This turn-based confirmation flow means a merchant's carefully-planned month can never be silently clobbered by a fresh generation request.

Editing on the grid: The merchant can click any day to add a new entry, click an existing entry to edit its topic/channel/format/audience/description in a modal, delete it, or simply drag it to a different date to reschedule. All of this happens live on the frontend before anything is persisted.

Saving - and the "why aren't I saved" bug that got fixed: When the merchant clicks **Save calendar**, two things happen in sequence:

1. The full month's entries are written to permanent storage - this is the definitive save.
2. The *working draft* sitting in the current chat session is silently cleared. This matters because as long as the session holds unsaved calendar content, the merchant is locked out of switching to a different Max agent - the system assumes there's unfinished work. Once the permanent save has it, though, there's nothing left to protect, so the lock should release.

Originally, clearing that session draft was itself routed through the normal chat-completion pipeline - meaning saving the calendar posted a visible "Cleared my content calendar draft" message in the chat, and worse, sometimes triggered an empty error reply from Max. This was wrong on two counts: clearing a technical/session detail is not something that should ever appear as a chat message, and it shouldn't be able to

fail loudly when the actual save already succeeded. The fix separated these concerns entirely: the permanent save is the operation that matters and is the only one that can show a real error if something goes wrong; the session-context clear became a silent, best-effort, backend-only action that never posts a chat message and never blocks the save from succeeding even if it itself fails. If that clear step can't find an active session (for example, if the merchant already closed the panel), it treats that as a no-op rather than an error - because a missing session simply means there's nothing left to clear.

The standalone dashboard page: Because a content calendar is something merchants want to check outside of a chat conversation, the exact same calendar is also available as a full-page, chat-independent view, reachable from the main dashboard side navigation under **Marketing → Content Calendar**. It loads and saves against the same permanent storage, so anything planned in the chat widget shows up here too, and vice versa - one calendar, two entry points.

3. Email Drafting & Multi-Email Campaigns

This feature lets Max write one email, or an entire multi-email campaign, with each draft kept safely independent.

How a merchant triggers it: *"Write me a promotional email for our fresh eggs"* for a single email, or *"Write me a 3-email campaign for our weekend sale"* for a series. Max opens the panel on the **Email** tab.

Drafting: Max writes a subject line and a body (simple formatting only - paragraphs, bold/italic, lists, links, and images from real, verified sources; never invented placeholder links or images). Critically, drafting does not overwrite - it **appends**. Every new draft is added to a running list rather than replacing whatever was there. This was a deliberate

redesign: earlier, generating a second email wiped out the first, which was frustrating when a merchant asked for a whole campaign and only got to see the last email written. Now, each new draft is pushed onto the list, and the panel jumps straight to showing the newest one.

Browsing drafts - the carousel: When there's more than one draft, the Email tab shows a small counter - "Email 1 of 3" - with left/right arrows. Clicking through moves between drafts; each is entirely independent, with its own subject, body, and editing state. Going back to draft 1 after editing draft 3 shows draft 1 exactly as it was. Editing one draft never touches the others.

The branded preview: Every draft renders in a live, fully-branded preview inside an isolated frame - the merchant's actual store logo (or a custom-uploaded header image, resizable via a slider), the brand's colors, and a "Visit our store" call-to-action button that links to the real store. Max never needs to add its own store links in the body text - the branded shell handles that automatically, so Max is instructed to write clear calls-to-action in words ("Shop the sale") and let the branded button carry the actual link.

Personalization: A merchant can click "Insert name" while editing the body, which drops in a placeholder token at the cursor. This is a platform-level merge token - at actual send time, it's swapped out per recipient with their real first name. In the in-app preview (but not the real sent email), the token is shown as a small styled pill so it's visually obvious where personalization will land.

Sending: Clicking Send opens a recipient picker - search and select from the merchant's contact list, or manually add one-off addresses - with a running count against the merchant's daily send limit, and a final "are you sure" confirmation step before anything actually goes out.

Clearing a draft - and why it's silent: Each draft has a **Clear** button (replacing what used to be a redundant "Save email" button - since drafts already live in the session automatically, there was nothing meaningful left for a manual save to do). Clicking Clear removes just that one draft from the list, leaving the others untouched. Like the calendar's clear, this action is deliberately silent - no chat message, no risk of an agent reply misfiring. Clearing the very last remaining draft empties the tab entirely and releases the agent-switch lock, since there's no unsaved email content left to protect.

4. Save Email to Calendar - Linking Campaigns to Dates

This feature bridges the Email and Content Calendar features: any drafted email can be pinned to a specific date, becoming a full calendar entry that carries the actual email content.

Setting up the date automatically: When Max drafts a multi-email campaign, it can attach a suggested date to each individual draft - spacing a 3-email campaign across, say, three specific dates in the month. This isn't required, but when present, it means the merchant doesn't have to think about scheduling at all.

Pinning a draft: On any email draft (in preview mode), the merchant clicks **Save to Calendar**. A small modal opens with the date field already filled in - using the draft's suggested date if Max set one - along with a topic pre-filled from the email's subject line, and the channel/format locked to "Email" (since this is unambiguously an email-type calendar entry). The merchant can adjust the date, add an audience note or description, then confirm.

What actually gets saved: Behind the scenes, the current month's existing calendar entries are fetched, a new entry is built that includes the standard fields (date, topic,

channel, format, audience, description) *plus* the actual email content, and either replaces an existing entry on that date or appends a new one. The whole month is then saved back in one call. This is purely a merchant-driven action - Max isn't the one deciding to schedule a send, the merchant is.

Revisiting a scheduled email: Later, opening the Content Calendar (either the widget tab or the standalone dashboard page) and clicking that day's entry shows the usual edit fields, plus a collapsible **"Preview email ▾"** section. Expanding it renders the actual branded email exactly as it would look when sent, with an inline rich-text editor underneath so the merchant can tweak the subject or body without ever leaving the calendar. Saving the calendar afterward persists any changes made here too, since the email content is just an ordinary part of the entry that flows through the same save path as everything else.

A note on scope: This feature stops at *scheduling and organizing* - it does not currently send the email automatically at the scheduled date/time. Building true automated scheduled-send would require additional infrastructure for reliable, at-scale delivery, which was deliberately descoped in favor of the simpler, immediately useful version: a calendar that visually tracks which emails are planned for which days, fully editable, with the actual send still done manually via the existing Send flow.

5. Social Media Captions

A lighter-weight feature: generating several ready-to-use caption options for a social post, rather than one generic line.

How a merchant triggers it: *"Write me Instagram captions for our fresh eggs."* Max opens the panel on the **Captions** tab.

Choosing an angle first: If the merchant hasn't specified a tone or angle, Max doesn't guess blindly - it first offers a small set of angle choices (e.g. "Playful," "Educational," "Urgency") as tappable suggestions, and waits for the merchant to pick one before writing anything. This keeps the generated captions actually usable rather than a generic miss.

Generating the options: Once the angle is known (or was already clear from the request), Max writes a topic and a short list of 2–3 caption variants. These render as individual cards on the tab, each showing a small icon for the target platform. Every card is independently copyable with a single click - the merchant picks whichever reads best and pastes it straight into their post.

6. Product Promotion

Rather than writing generic promotional copy, Max can build content directly around a real product from the merchant's catalog.

How a merchant triggers it: By attaching or referencing a specific product in their request - e.g. *"Generate an email to promote my Farm Eggs (12 pcs)"* with the product attached as a resource. Max first reads the merchant's actual product data - name, price, and primary image - rather than inventing details.

Building the content: With the real product data in hand, Max then produces whichever type of content matches the request - a promotional email (embedding the real product photo in the branded layout), social captions about that product, or a calendar entry featuring it. The underlying rule enforced throughout the system: Max may only use image URLs and product details actually verified from the merchant's own catalog -

never a guessed, placeholder, or invented image/link. This keeps every product-promotion output grounded in what's actually in the merchant's store.

Shared Infrastructure Across All Features

A few mechanisms are common to every tab and worth calling out on their own, since they explain a lot of the "why" behind how the system behaves:

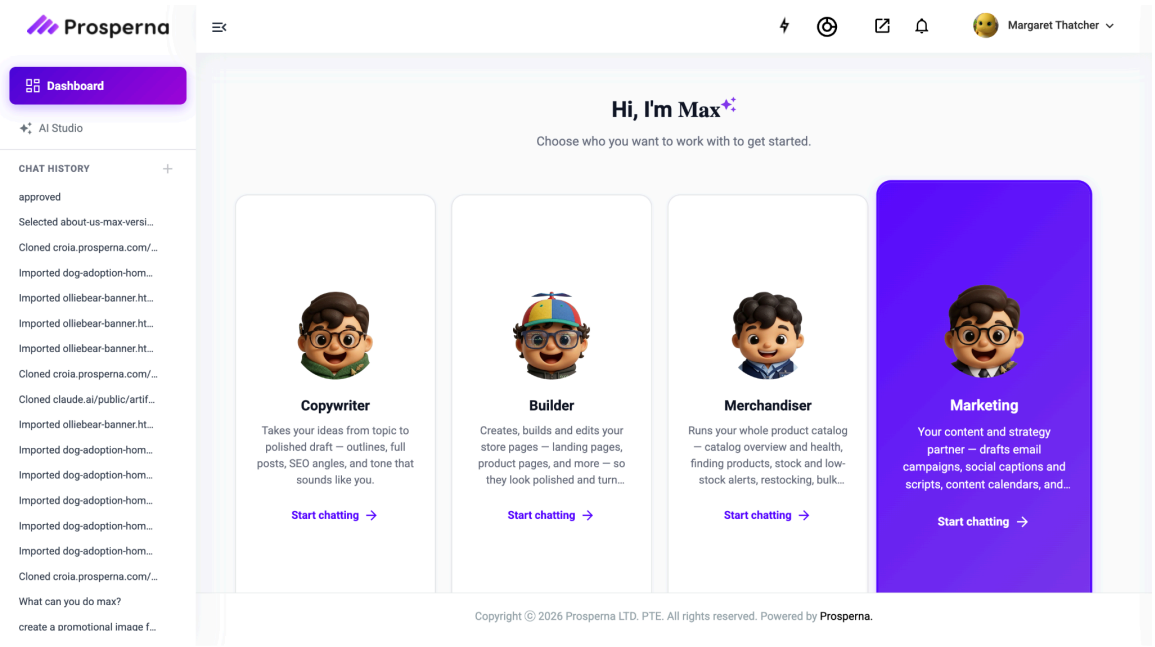
One panel, one entry point: Every feature uses the same mechanism to open the Marketing panel and jump to the right tab. If the panel is already open, it doesn't discard whatever's in progress - it just switches the active tab and, for the calendar, refreshes that month's data - so a merchant mid-way through drafting an email doesn't lose that work by asking for captions in the same breath.

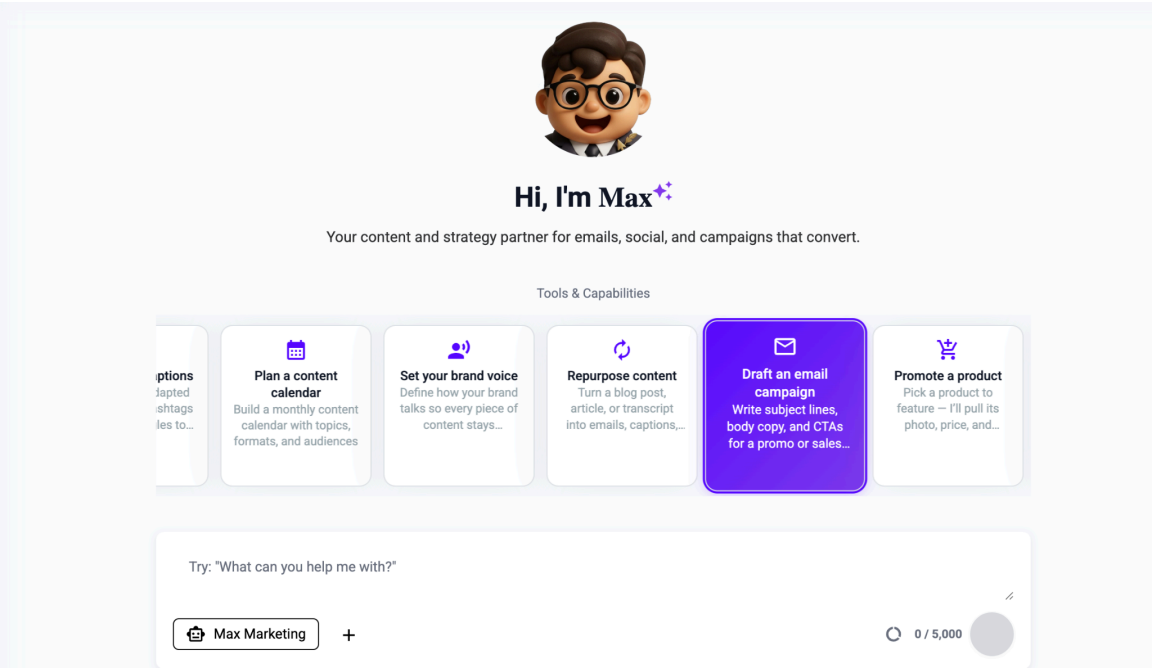
Session context vs. permanent storage: Everything in the Marketing panel lives, moment to moment, in the current chat session - this is what lets the panel re-populate correctly if the merchant navigates away and back within the same conversation. But only some of that context is meant to be permanent. Brand voice and the content calendar are written through to real durable storage. Email drafts and captions are intentionally session-only - they're working material meant to be copied out, sent, or (for email) optionally pinned to the calendar, not stored as standing preferences.

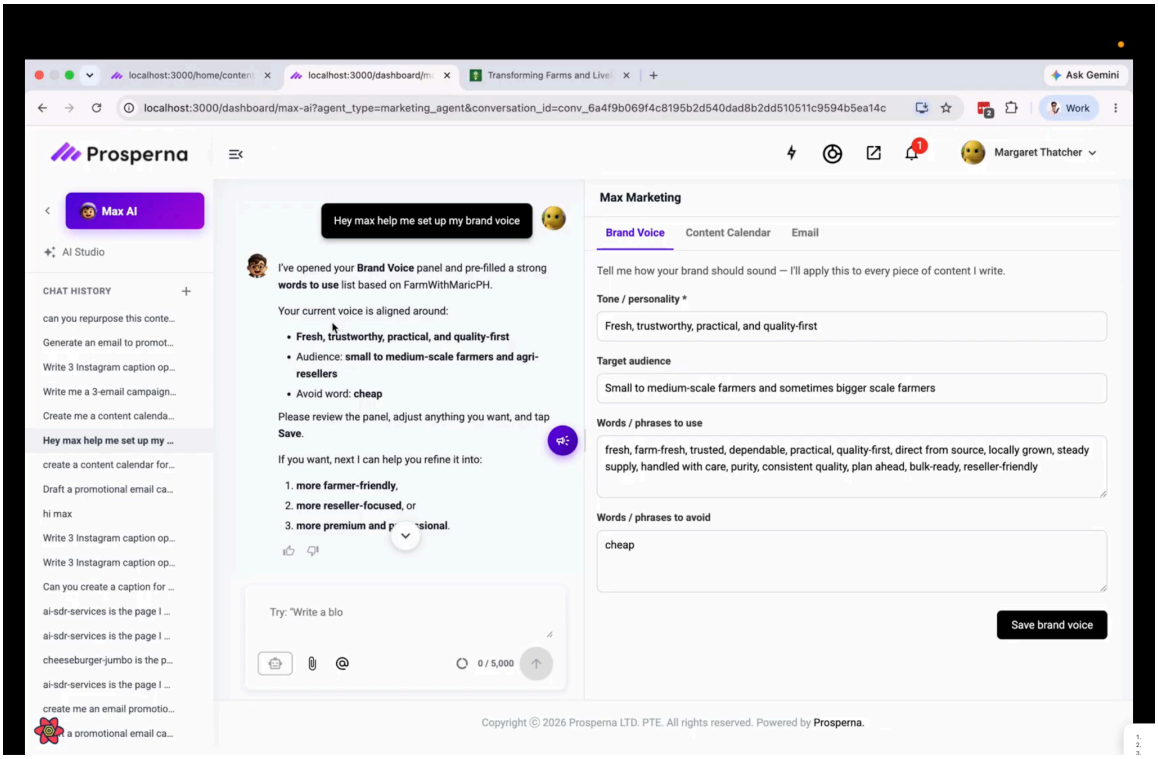
The agent-switch lock: Because a merchant might have unsaved work sitting in the Marketing panel, the system blocks switching to a different Max agent while there's genuinely unsaved content - an email draft not yet cleared, calendar entries not yet saved, or in-progress caption options. This is intentionally scoped tightly: brand voice, once saved, never contributes to this lock (it's persisted, not transient), and saving the

calendar or clearing an email draft actively releases the lock rather than requiring the merchant to hunt for some separate "unlock" action.

Silent housekeeping, visible work: A clear design principle that emerged through iteration: anything that's purely bookkeeping - clearing a stale draft from the session, dropping a working calendar draft after it's already safely saved permanently - should never generate a visible chat message or be capable of producing a visible error. Only genuine content actions (drafting an email, planning a calendar, writing captions) and genuine failures (the permanent save itself failing) surface to the merchant. This was reached after finding the opposite pattern caused real, confusing bugs: an internal session cleanup posting a "Cleared..." message and then Max replying with a blank error, even though the actual save had succeeded.

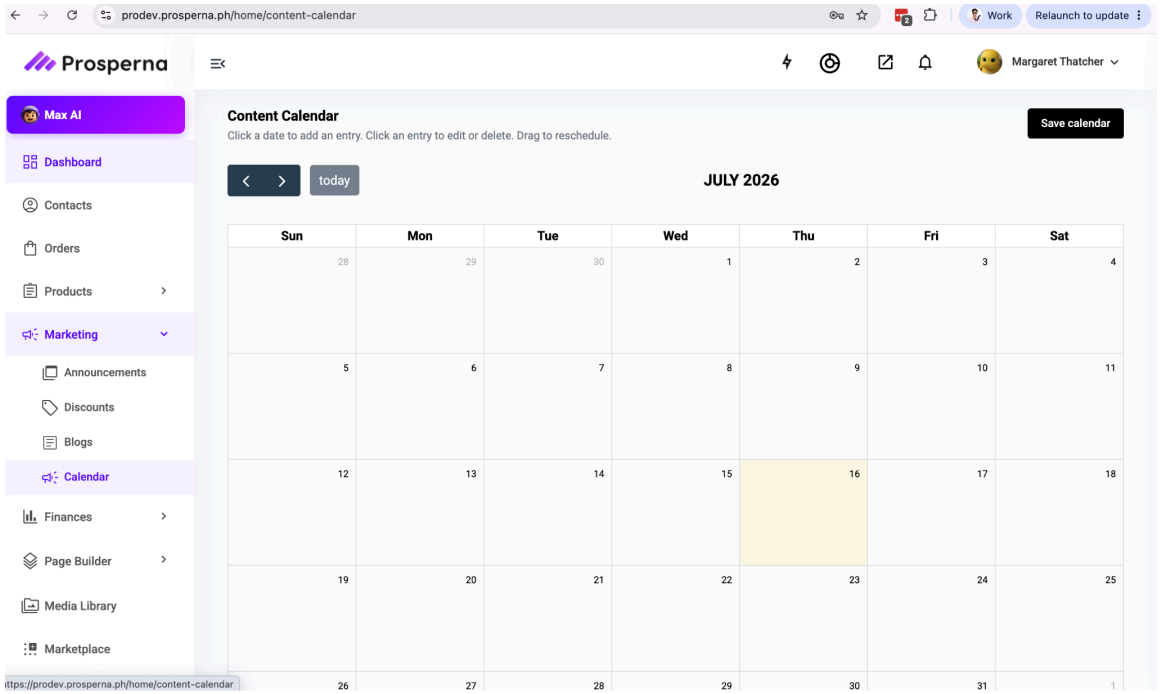


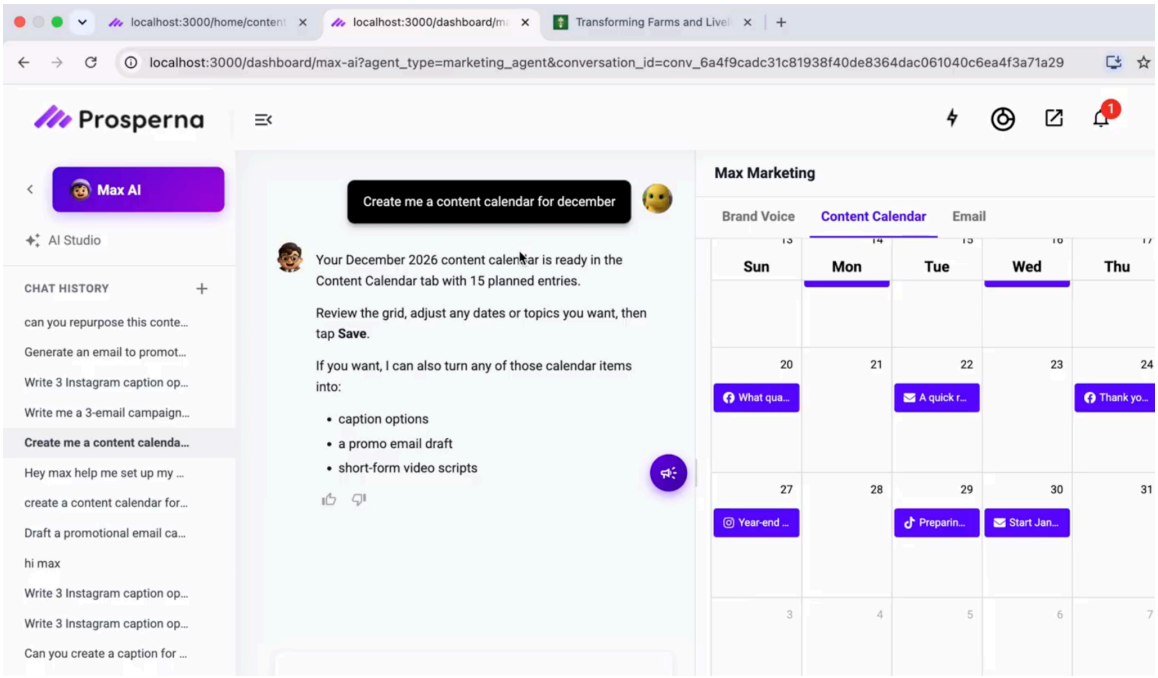

Hi, I'm Max
Your content and strategy partner for emails, social, and campaigns that convert.
Tools & Capabilities
Plan a content calendar
Build a monthly content calendar with topics, formats, and audiences
Set your brand voice
Define how your brand talks so every piece of content stays...
Repurpose content
Turn a blog post, article, or transcript into emails, captions,...
Draft an email campaign
Write subject lines, body copy, and CTAs for a promo or sales...
Promote a product
Pick a product to feature — I'll pull its photo, price, and...
Try: "What can you help me with?"
Max Marketing
0 / 5,000

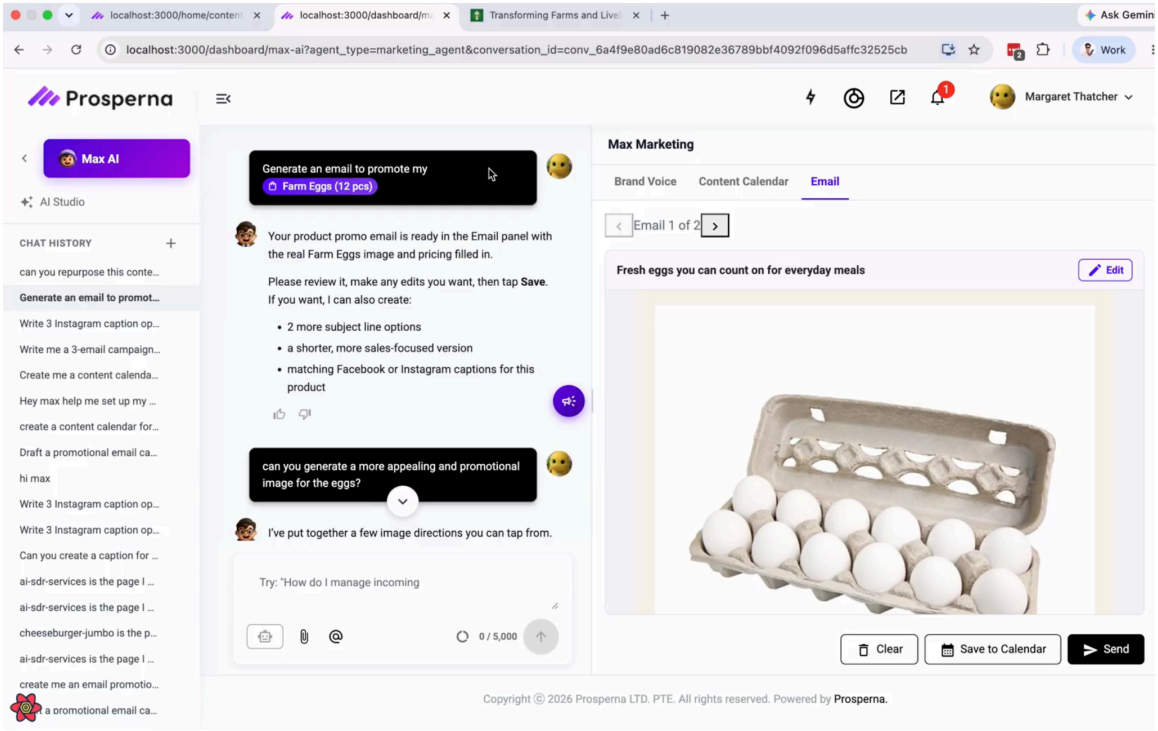

Prosperna
Max AI
AI Studio
CHAT HISTORY
Hey max help me set up my brand voice
I've opened your Brand Voice panel and pre-filled a strong words to use list based on FarmWithMaricPH.
Your current voice is aligned around:
Fresh, trustworthy, practical, and quality-first
Audience: small to medium-scale farmers and agri-resellers
Avoid word: cheap
Please review the panel, adjust anything you want, and tap Save.
If you want, next I can help you refine it into:
1. more farmer-friendly,
2. more reseller-focused, or
3. more premium and p sional.
Max Marketing
Brand Voice
Content Calendar
Email
Tell me how your brand should sound — I'll apply this to every piece of content I write.
Tone / personality *
Fresh, trustworthy, practical, and quality-first
Target audience
Small to medium-scale farmers and sometimes bigger scale farmers
Words / phrases to use
fresh, farm-fresh, trusted, dependable, practical, quality-first, direct from source, locally grown, steady supply, handled with care, purity, consistent quality, plan ahead, bulk-ready, reseller-friendly
Words / phrases to avoid
cheap
Save brand voice
Try: "Write a blo
0 / 5,000
Copyright © 2026 Prosperna LTD. PTE. All rights reserved. Powered by Prosperna.

Content Calendar System

Built persistent content calendar with full-page editor and market service integration. Added MongoDB schema for calendar entries with email scheduling, subject/body fields, and template support. Implemented email preview functionality and media attachment support. Integrated with marketing widget for email saving and management.







Abandoned Cart Reminder System

Designed and implemented system architecture for detecting and tracking abandoned shopping carts. Created background job for cart abandonment detection at configurable time intervals.

Built customer communication templates for email and SMS reminders. Integrated with existing order and notification systems.

System Improvements & Optimizations

Enhanced navigation with dashboard linking and per-agent MaxAI hints. Improved email rendering and styling across browsers. Fixed database indexing for sparse index operations. Resolved merge conflicts across multiple development branches. Improved test coverage and code quality.

[Note: Screenshots of modules and UI components will be inserted here]

2.4 Technical Achievements

1. React Component Architecture: Designed and implemented complex components with proper state management and performance optimization
2. Backend API Development: Created RESTful endpoints with proper error handling, validation, and integration with existing services
3. Database Schema Design: Implemented MongoDB schemas with proper indexing strategies and query optimization
4. Full-Stack Integration: Successfully integrated frontend UI with backend APIs, AI pipelines, and existing service infrastructure
5. Bug Resolution: Identified and fixed 15+ critical and major bugs affecting production systems
6. Code Quality: Maintained high code standards through 44 commits with clear messages and comprehensive testing
7. Merge Conflict Management: Resolved 5+ complex merge conflicts across concurrent feature branches

3. Synthesis of the Practicum Engagement

3.1 Learnings Gained

Full-Stack Development at Scale

- Implemented complete features from database to UI across multiple microservices. Gained deep understanding of how frontend and backend components interact in production systems supporting thousands of users.

React Architecture & Performance

- Mastered complex React component patterns, state management with hooks, and performance optimization techniques. Learned to identify and fix issues like white-screen crashes from stale closures and tab-switch cache problems.

Backend API Design

- Developed RESTful APIs with proper error handling, validation, and integration patterns. Understood the importance of clear API contracts for frontend-backend coordination.

Database Optimization

- Gained expertise in MongoDB schema design, indexing strategies, query optimization, and the impact of database constraints on API design. Learned about sparse indexes and their behavior.

AI/ML Pipeline Integration

- Understood how to integrate new data sources into existing AI pipelines. Learned that proper data formatting and field extraction are crucial for AI model effectiveness.

Debugging Complex Systems

- Developed systematic debugging approaches for multi-system interactions. Learned to use logging strategically and trace data flow across components.

Professional Development Practices

- Experienced rigorous code review culture, collaborative development, version control workflows, and the importance of clear commit messages and documentation.

Soft Skills Development

- Improved communication skills through PR reviews, team discussions, and technical documentation. Learned to explain complex technical decisions to peers.

3.2 Realizations

8. Small bugs in data extraction or field identification have cascading effects across entire systems, requiring careful attention to detail.
9. Production systems require systematic debugging and problem-solving approaches that go far beyond what's possible in test environments.
10. Code review and peer feedback are critical for identifying edge cases and improving overall solution quality.
11. Integration testing with actual data flows reveals issues that unit tests cannot catch.
12. Clear commit messages and focused commits are invaluable for understanding changes and debugging issues later.
13. Database decisions (schemas, indexing, constraints) have significant impact on API design and overall system performance.

14. AI/ML systems require carefully formatted, consistently extracted input data to function effectively.
15. Communication and documentation are as important as coding skills in professional software development.

3.3 Conclusion

The practicum engagement at Prosperna Inc. provided invaluable hands-on experience in professional software engineering. Working across multiple divisions, repositories, and technologies reinforced the importance of adaptability, systematic problem-solving, and collaborative development practices.

The commits across 8 repositories represent real contributions to production systems serving thousands of users. From implementing the Pippify web form system that enables non-technical users to create lead capture forms, to developing the abandoned cart reminder system that helps businesses recover lost sales, the work performed has tangible business value.

This engagement has significantly accelerated technical growth and provided insights into real-world software development that extend far beyond classroom learning. The experience working with experienced engineers, navigating production systems, and delivering features that directly impact users has established a strong foundation for a successful career in software engineering. Key takeaways include the importance of thorough testing, clear communication, and a commitment to code quality. The collaborative environment at Prosperna and the mentorship received from senior engineers have been invaluable in developing both technical and professional skills.

Appendix A

Competency-Based CV

JOHN ALDRINE F. LIM

Brgy., Balibago, Sta. Rosa City Laguna | (63+) 995 726 9133 | johnaldrine.lim.04@gmail.com
www.linkedin.com/in/john-aldrine-lim-a75a442ba/ | https://github.com/dev-aldrine

TECHNICAL SKILLS

Languages: Python, Java, C#, PHP
Tools | Frameworks: Git, Postman, Docker, Figma, VS code, Supabase, XAMPP, hugging face, Apache
Data Analysis | Machine Learning: Matplotlib, Scikit-learn, Pytorch, NumPy, Pandas
Frontend: HTML, CSS, JS, Bootstrap, Tailwind
Backend and Database: gradio, Flask, ASP.NET - MySQL, PostgreSQL
AI Tools: ChatGPT, Gemini, V0, Claude, OpenClaw

EDUCATION

Mapúa Malayan Colleges Laguna - Bachelor of Computer Science (GWA as of 3rd year: 1.5) **Expected:** 2026

- *Dean's Lister: 1st year (SY 2022 - 2023) – 3rd year (SY 2024 - 2025)*
- *President's Lister: 2nd year (SY 2023 - 2024)*

Specialization: Machine Learning
Relevant Coursework: Machine Learning Theory, Neural Network, Discrete Structures and Algorithms, OOP Concepts, Web Systems and Technologies, Software Engineering, Automata Theory, Computational Learning Theory

PROJECTS

Resume Analyzer Web App – ***Python, Flask, HTML/CSS/JS***

- Developed a web-based system to analyze resumes and extract key skills using Python and Gemini API.
- Implemented Flask backend to handle user submissions and display results dynamically.
- Built frontend with HTML/CSS/JavaScript for a clean, interactive interface.

Machine Learning Model for Price Prediction – ***Python, Flask, scikit-learn, NumPy, pandas, leaflet***

- Built a predictive model for commodity pricing using categorical and geolocation features.
- Implemented feature scaling, early stopping, and model saving/loading.
- Evaluated model performance with test MSE to ensure accuracy.

CNN Model for Predicting Ink level Categories – ***Python, Flask, PyTorch, gradio***

- Built a model that automatically classifies ink usage of PDF pages into light, medium, heavy categories.
- Flask backend to handle PDF uploads, extract images of pages, and serve predictions from the model.
- Used Gradio to host the model as an API on Hugging Face, reducing load on the main backend server.

Traffic Control Using Computer Vision – ***Python, Haar Cascades***

- Developed a traffic monitoring system detecting civilians and cars in real-time using Haar Cascade classifiers.
- Implemented Python scripts to track movement and support automated traffic analysis.

CERTIFICATES

Google Cloud Skill Badges: *Prepare Data for ML APIs • Set Up an App Dev Environment • Build a Secure Google Cloud Network • Implementing Cloud Load Balancing for Compute Engine* *Accomplished in July 2024*
Google Cloud Computing Foundations: *Data, ML, and AI • Infrastructure in Google Cloud • Networking & Security • Cloud Computing Fundamentals* *Accomplished in July 2024*
TOEIC Speaking and Writing: Proficiency level 7 | **TOEIC Listening and Reading:** 855/900 *Valid until June 1, 2027*

SEMINARS

Technopreneurship: A Journey in Building Your Own Start Up *March 2024*

ORGANIZATIONS

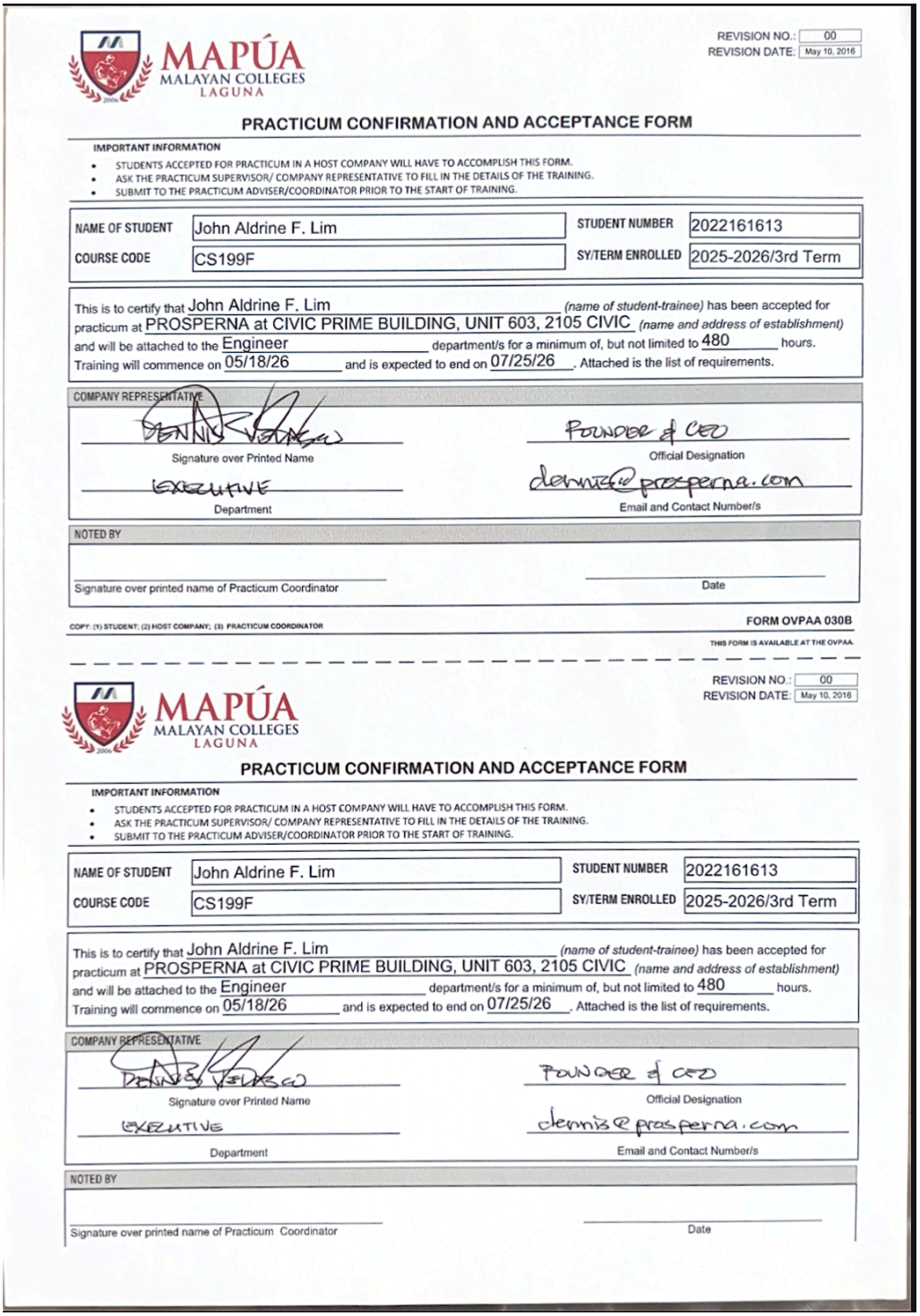
Junior Philippine Computer Society (JPCS) – Member *2023 – Present*
Junior ACM – Member *2021 - 2022*

Appendix B

Endorsement Letter

Appendix C

Practicum Acceptance



MAPÚA MALAYAN COLLEGES LAGUNA

REVISION NO.: 00
REVISION DATE: May 10, 2016

PRACTICUM CONFIRMATION AND ACCEPTANCE FORM

IMPORTANT INFORMATION

- STUDENTS ACCEPTED FOR PRACTICUM IN A HOST COMPANY WILL HAVE TO ACCOMPLISH THIS FORM.
- ASK THE PRACTICUM SUPERVISOR/ COMPANY REPRESENTATIVE TO FILL IN THE DETAILS OF THE TRAINING.
- SUBMIT TO THE PRACTICUM ADVISER/COORDINATOR PRIOR TO THE START OF TRAINING.

NAME OF STUDENT	John Aldrine F. Lim	STUDENT NUMBER	2022161613
COURSE CODE	CS199F	SY/TERM ENROLLED	2025-2026/3rd Term

This is to certify that John Aldrine F. Lim *(name of student-trainee)* has been accepted for practicum at PROSPERNA at CIVIC PRIME BUILDING, UNIT 603, 2105 CIVIC *(name and address of establishment)* and will be attached to the Engineer department/s for a minimum of, but not limited to 480 hours. Training will commence on 05/18/26 and is expected to end on 07/25/26. Attached is the list of requirements.

COMPANY REPRESENTATIVE

DENNIS [illegible]
Signature over Printed Name

FOUNDER & CEO
Official Designation

EXECUTIVE
Department

dennis@prosperna.com
Email and Contact Number/s

NOTED BY

Signature over printed name of Practicum Coordinator

Date

COPY: (1) STUDENT; (2) HOST COMPANY; (3) PRACTICUM COORDINATOR

FORM OVPAA 030B

THIS FORM IS AVAILABLE AT THE OVPAA.

MAPÚA MALAYAN COLLEGES LAGUNA

REVISION NO.: 00
REVISION DATE: May 10, 2016

PRACTICUM CONFIRMATION AND ACCEPTANCE FORM

IMPORTANT INFORMATION

- STUDENTS ACCEPTED FOR PRACTICUM IN A HOST COMPANY WILL HAVE TO ACCOMPLISH THIS FORM.
- ASK THE PRACTICUM SUPERVISOR/ COMPANY REPRESENTATIVE TO FILL IN THE DETAILS OF THE TRAINING.
- SUBMIT TO THE PRACTICUM ADVISER/COORDINATOR PRIOR TO THE START OF TRAINING.

NAME OF STUDENT	John Aldrine F. Lim	STUDENT NUMBER	2022161613
COURSE CODE	CS199F	SY/TERM ENROLLED	2025-2026/3rd Term

This is to certify that John Aldrine F. Lim *(name of student-trainee)* has been accepted for practicum at PROSPERNA at CIVIC PRIME BUILDING, UNIT 603, 2105 CIVIC *(name and address of establishment)* and will be attached to the Engineer department/s for a minimum of, but not limited to 480 hours. Training will commence on 05/18/26 and is expected to end on 07/25/26. Attached is the list of requirements.

COMPANY REPRESENTATIVE

DENNIS [illegible]
Signature over Printed Name

FOUNDER & CEO
Official Designation

EXECUTIVE
Department

dennis@prosperna.com
Email and Contact Number/s

NOTED BY

Signature over printed name of Practicum Coordinator

Date

Appendix E

Training Plan

MAPÚA MALAYAN COLLEGES LAGUNA

REVISION NO.: 00
REVISION DATE: May 10, 2016

TRAINING PLAN

NAME	John Aldrine F. Lim	COURSE CODE	CS199F
PROGRAM & STUDENT NO.	BSCS 2022161613	COURSE TITLE	CS PRACTICUM

STUDENT OUTCOMES

CO1. Identify, analyze, and recommend solution to the computing problem being faced by the organization.
CO2. Apply the different concepts in Computer Science in dealing with the problem-solving process of the organization, and
CO3. Acquire new knowledge and experience while in the organization.

AREAS / PHASES OF TRAINING AND TIME ALLOTMENT

1. Onboarding & Environment Setup (32 hours)
2. Technical Foundation: AI/ML + System Basics (68 hours)
3. Core Development: Frontend & Backend (100 hours)
4. AI Integration & Data Handling (100 hours)
5. Advanced Development & Optimization (100 hours)
6. Capstone & Delivery (80 hours)

EVALUATION GUIDELINES & COURSE OUTCOMES

DEMONSTRATION OF SOFT SKILLS (40%)

KEY AREAS

COMMUNICATION SKILLS (20%)

Relate to co-trainees/supervisors terminologies and rules
Recite procedures and instructions needed for the tasks
Identify and describe safety signs and symbols
Ask critical questions related to the tasks
Produce well-written regular and incident reports
Prepares and presents reports using Information and Communication Technology (ICT)

PROFESSIONAL DEPORTMENT (20%)

Observes proper grooming and attire
Reports to work regularly on time and as necessary, even beyond prescribed working hour
Acts according to the job description given by the company
Willing to accept new tasks apart from the usual routine and responsibilities
Delivers quality output on time
Demonstrates respect for different individuals

INITIATIVE (+5%)

Volunteers to perform tasks beyond routine tasks

DEMONSTRATION OF TECHNICAL SKILLS (60%)

Key Areas

SOFTWARE DEVELOPMENT SKILLS (40%)

- Able to independently build end-to-end features spanning frontend UI, backend APIs, and database design (20%)
- Able to write clean, tested, documented code that passes senior engineer review with minimal revisions (20%)

AI & SYSTEMS INTEGRATION SKILLS (20%)

- Able to successfully integrate and optimize different AI/ML technologies in production contexts (10%)
- Able to debug complex issues independently using logs, profiling tools, and systematic troubleshooting approaches (10%)

INITIATIVE (+5%)

Volunteers to perform tasks beyond routine tasks

CONFORME	CONSENT (FOR MINORS ONLY)	NOTED BY	ENDORSED BY	APPROVED BY
SIGNATURE OVER PRINTED NAME OF STUDENT / DATE	SIGNATURE OVER PRINTED NAME OF PARENT OR GUARDIAN / DATE	DENNIS VELASCO SIGNATURE OVER PRINTED NAME OF PRACTICUM SUPERVISOR / DATE	SIGNATURE OVER PRINTED NAME OF PRACTICUM ADVISER / DATE	SIGNATURE OVER PRINTED NAME OF PROGRAM CHAIR / DATE

COPY: (1) STUDENT; (2) HOST COMPANY; (3) PRACTICUM COORDINATOR

FORM OVPAA-030D
THIS FORM IS AVAILABLE AT THE OVPAA.